

Additional file 1 — Supplementary Information

Additional file 1 — Supplementary Information

Provenance and reproducibility

S1.1 Guardian validator specifications

S1.2 Programmatic access

S1.3 Additional validation on standard R datasets

S1.4 Guardian test-suite coverage

S1.5 Performance benchmarks

References cited in this Additional file

Additional file 1 — Supplementary Information

StickForStats: automated statistical assumption validation for reproducible computational biology

Vishal Bharti, Debojyoti Chakraborty

Provenance and reproducibility

Every number in this document was produced by executing code in the repository, not by transcription from any prior document. All values below were emitted at commit **41d4a27e5fb0e9d221c493db5f5b2e0d4df17005** (release **v1.2.0**, archived on Zenodo as 10.5281/zenodo.21797621) of https://github.com/visvikbharti/stickforstats_new, on macOS 26.5.2 (arm64), with **Python 3.11.11**, **NumPy 2.4.6**, **SciPy 1.17.1**, **pandas 2.3.3**, **scikit-learn 1.9.0**, **statsmodels 0.14.6**, and **R 4.4.1 (2024-06-14)** for the reference datasets and the independent cross-checks. The declared random seed is **20260804**; sections S1.1, S1.3 and S1.4 are deterministic (no random numbers are drawn), and the one seeded example in S1.2 states its seed inline. Sections S1.1, S1.3 and S1.4 are regenerated in full by

```
DJANGO_SETTINGS_MODULE=stickforstats.settings \  
  .venv-django/bin/python paper/replication/additional_file_1_evidence.py
```

which prints the commit hash, the working-tree status, every library version and every number reproduced below. S1.2 is a set of shell/Python transcripts, reproduced by the commands quoted verbatim in that section. S1.5 is regenerated by `paper/replication/benchmark_api.py`. Where a value can be checked against an independent implementation (R, statsmodels, exact rational arithmetic) that check is shown alongside it.

One editorial substitution is made for readability and is flagged here so the transcripts can be taken literally otherwise: the S1.2 examples were captured against a local `manage.py runserver` instance bound to port 8077 (to avoid a clash with another process), whereas the quoted commands show the documented default base URL `http://localhost:8000/api/v1`. Only that URL string was changed; no output line, and no number anywhere in this document, was edited by hand.

S1.1 Guardian validator specifications

Regenerated by: `paper/replication/additional_file_1_evidence.py --only s1_1`

S1.1.1 The nine validators

The validator registry is `GuardianCore.validators` in `backend/core/guardian/guardian_core.py`. Executed, `len(GuardianCore().validators) == 9`. The table is the registry itself, key by key.

#	Registry key	Implementing class	Assumption checked	Statistical method
1	normality	NormalityValidator	Normality of each group's values	Shapiro–Wilk for $n \leq 5000$; Anderson–Darling for $n > 5000$, converted to a continuous p -value by the D'Agostino–Stephens (1986) closed form. Severity is critical if any violating group has $p < \alpha/10$, else warning.
2	variance_homogeneity	VarianceHomogeneityValidator	Equality of variances across groups	Levene's test with center="median" (Brown–Forsythe). Severity from the max/min variance ratio: > 4.0 critical, > 2.0 warning, else minor (Box, 1954).
3	independence	IndependenceValidator	Serial dependence in observation order	Lag-1 Pearson autocorrelation on the observation series, with the inferential p -value from the Pearson test. Gated: it is computed only when the caller declares observation_order to be one of sequential, temporal, time, timeseries, time_series, time-series, ordered, serial. Otherwise the check is recorded as not_applicable and referred to study design.
4	outliers	OutlierDetector	Presence of extreme values	Union of Tukey IQR fencing ($Q1 - 1.5 \cdot IQR$, $Q3 + 1.5 \cdot IQR$) and $ z > 3$, computed per group. Severity from the reported percentage: $> 10\%$

#	Registry key	Implementing class	Assumption checked	Statistical method
				critical, > 5% warning, else minor. The reported percentage is the mean over groups containing at least one fenced value (see S1.3.3). Thresholds are fixed constants: <code>validate()</code> accepts α but does not use it.
5	sample_size	SampleSizeValidator	Adequacy of n for the requested test	Fixed thresholds on the smallest group: $n < 3 \rightarrow$ critical, $n < 20 \rightarrow$ warning, else satisfied. Executed, these are the same for every test type — the validator does not receive the test type — and α is accepted but unused.
6	modality	ModalityDetector	Multimodality	Peak counting on a Gaussian kernel density estimate (<code>scipy.stats.gaussian_kde</code> , default Scott's-rule bandwidth) evaluated on a 50-point grid; groups with $n < 20$ are skipped. Flags warning when more than one local maximum exceeds 30% of the tallest peak. This is a descriptive heuristic, not an inferential test: it returns no statistic and no p -value, and it is not Hartigan's dip test.
7	linearity	LinearityValidator	Linearity of the X–Y relationship	Requires exactly two arrays. Compares R^2 of a

#	Registry key	Implementing class	Assumption checked	Statistical method
				linear against a degree-2 polynomial fit, plus a Wald–Wolfowitz runs test on the linear-fit residuals. The returned statistic is the R^2 improvement; the returned p_value is the runs-test p . Violation if the runs test detects a pattern (critical), or if the R^2 improvement > 0.10 (critical) or > 0.05 (warning). There is no RESET test: grep -ri reset backend/core/guardian/ returns no match.
8	homoscedasticity	HomoscedasticityValidator	Constant residual variance	Breusch–Pagan: regress squared OLS residuals on X , statistic $n R^2$, referred to $\chi^2(1)$. Requires exactly two arrays and $n \geq 10$.
9	similar_shapes	SimilarShapesValidator	Common distribution shape across groups, for the rank-based location tests	Two-sample Kolmogorov–Smirnov on median-centred groups: each group has its own median subtracted, removing the location shift the rank test is about, and the centred empirical distributions are compared. For $k > 2$ groups all $k(k-1)/2$ pairs are compared against a Bonferroni-corrected α . Severity uses the KS statistic D as

#	Registry key	Implementing class	Assumption checked	Statistical method
				well as p : warning if $D > 0.161$, else minor. Never critical — see below. Groups with $n < 5$ cannot be compared, and that case returns an explicit warning recording the assumption as <i>unverified</i> , not as satisfied.

Independent verification of validators 1, 2 and 8 (executed; see S1.3 for the datasets):

Validator	StickForStats value	Independent implementation	Agreement
Normality (Shapiro–Wilk, ToothGrowth OJ)	$W = 0.9178433795430943$, $p = 0.023587729215913122$	R <code>shapiro.test</code> : $W = 0.91784337938922911$, $p = 0.023587728999479349$	9 significant figures
Variance homogeneity (Levene/Brown–Forsythe, PlantGrowth)	$p = 0.34122662412547355$	R <code>car::leveneTest(center = median)</code> : $F(2, 27) = 1.1192$, $p = 0.3412$	to R's printed precision
Homoscedasticity (Breusch–Pagan, mtcars mpg ~ wt)	statistic = 0.0404383955 , $p = 0.8406258537$	<code>statsmodels.stats.diagnostic.het_breuschpagan</code> : LM = 0.0404383955 , $p = 0.8406258537$	10 decimal places

For the Breusch–Pagan check, R's `lmtest::bptest(m, studentize = FALSE)` reports BP = 0.037942 , $p = 0.8456$ — a different variance normalisation of the same test, reaching the same conclusion.

Validators 1, 2 and 4 delegate their test statistics to SciPy (`stats.shapiro`, `stats.levene`, `stats.zscore/np.percentile`), so their numerical agreement with SciPy is exact by construction; the value of the Guardian layer is the gating, severity assignment and routing built on top of them, not a reimplementations of the statistics.

S1.1.2 Test-type-to-validator mapping (derived from the code)

The mapping is `GuardianCore.test_requirements`. Executed, it has **25** test-type keys (including aliases). The seven test types printed in manuscript Table 1 map as follows. This matrix is a direct dump of `test_requirements`, **not** a transcription of Table 1:

Test type (code key)	normality	variance_homogeneity	independence	outliers	sample_size	modality	linearity	homoscedasticity	similar_shapes
t_test	X	X	X	X					
anova	X	X	X	X					
pearson	X			X			X		
regression	X		X				X	X	
chi_square			X						
mann_whitney			X						X
kruskal_wallis			X						X

Verbatim requirement lists:

```

t_test      = ['normality', 'variance_homogeneity', 'independence',
'outliers']
anova       = ['normality', 'variance_homogeneity', 'independence',
'outliers']
pearson     = ['normality', 'linearity', 'outliers']
regression  = ['normality', 'independence', 'homoscedasticity', 'linearity']
chi_square  = ['expected_frequencies', 'independence']
mann_whitney = ['independence', 'similar_shapes']
kruskal_wallis = ['independence', 'similar_shapes']

```

Three qualifications that the column matrix cannot express, and that a reader checking the code will find:

1. **chi_square has a requirement that is not one of the nine validators.** Its first requirement is `expected_frequencies`, which has no entry in `GuardianCore.validators` and no column above. It is handled by a dedicated contingency-table code path (`_check_contingency`) implementing Cochran's expected-cell- frequency rule. It is *not* the `sample_size` validator, and should not be filed under a "sample size" column.
2. **mann_whitney and kruskal_wallis require similar_shapes, which is validator 9.** It was for a period declared in `test_requirements` with no implementation behind it: the dispatch loop reads `if req in self.validators`, so the requirement was skipped in silence while `assumptions_checked` still named it. Measured at the time, normal-versus-exponential groups, a hundredfold difference in spread, and strong bimodality against a unimodal comparison each returned zero violations and a confidence score of 1.000, with `similar_shapes` listed as an assumption checked. It is now implemented as a two-sample Kolmogorov–Smirnov test on median-centred groups (§S1.1.1, validator 9), and a construction-time invariant, `_assert_every_requirement_is_implemented`, now refuses to build a `GuardianCore` in which any declared requirement lacks either a registered validator or a dedicated code path — so the same silence cannot recur for the next requirement added.

3. **modality is declared but unreachable through test_requirements.** Executed, the set of registered validators never referenced by any test type is exactly ['modality']. It is available to direct callers but no test type activates it. sample_size is reachable, but only for factor_analysis and pca.
4. **manova is listed with univariate assumptions only.** Guardian evaluates normality, variance homogeneity, independence and outliers on the response values supplied. It does **not** evaluate multivariate normality of the response vector, nor equality of the group covariance matrices (Box's M); no validator implements either. The entry exists because the alternative was worse rather than because the coverage is complete: before it, manova was absent from the table, so the MANOVA interface received a report with zero checks and confidence 1.000 and displayed an all-clear having verified nothing.

Two design consequences worth stating plainly:

- For regression, the normality validator is applied to the **raw X and Y arrays**, not to the model residuals (the residual-based checks for regression are linearity and homoscedasticity). The reported p -value in a non-violating case is the **maximum** Shapiro–Wilk p across the supplied arrays; in a violating case it is the **minimum** p across the violating arrays.
- t_test requirements are design-aware at call time: for design="one_sample" or design="paired", variance_homogeneity is removed from the requirement list, and for design="paired" normality and outliers are assessed on the paired **differences** rather than on the raw columns.

Full mapping, all 25 keys:

anova	['normality', 'variance_homogeneity', 'independence', 'outliers']
bayesian_anova	['normality', 'independence']
bayesian_correlation	['independence']
bayesian_t_test	['normality', 'independence']
chi_square	['expected_frequencies', 'independence']
cox_regression	['independence']
did	['independence']
difference_in_differences	['independence']
factor_analysis	['normality', 'sample_size']
hlm	['normality', 'independence', 'homoscedasticity']
iv	['independence']
kruskal_wallis	['independence', 'similar_shapes']
lmm	['normality', 'independence', 'homoscedasticity']
mann_whitney	['independence', 'similar_shapes']
manova	['normality', 'variance_homogeneity', 'independence', 'outliers']
mediation	['normality', 'independence', 'linearity']
mixed_model	['normality', 'independence', 'homoscedasticity']
multilevel	['normality', 'independence', 'homoscedasticity']
pca	['sample_size']
pearson	['normality', 'linearity', 'outliers']
propensity_score	['independence']
psm	['independence']
regression	['normality', 'independence', 'homoscedasticity', 'linearity']
survival	['independence']


```
t_test          ['normality', 'variance_homogeneity', 'independence',  
'outliers']
```

S1.1.3 Severity weights, the confidence score, and what it can actually take

Severity weights are `SEVERITY_WEIGHTS = {critical: 3.0, warning: 2.0, minor: 1.0}` and the composite score is

$C = \max(0, 1 - \sum w_i / (W_{\max} \times 1.2))$, where $W_{\max} = k \times 3.0$ for k violations.

Because the violation count k appears in both numerator and denominator, it cancels: the score measures the **mean severity** of the violations, not how many there are.

Executed over uniform-severity violation sets of size 1, 2, 5 and 100:

Violation set	Confidence, executed
no violations	1.000
any number of minor only	0.722
any number of warning only	0.444
any number of critical only	0.167

So with at least one violation the score can never exceed 0.722, and the maximum attainable value in the “0.6–0.8 caution” band is 0.722. Routing is **not** decided by this score: it is decided by severity, via `can_proceed = len(critical_violations) == 0` (`guardian_core.py`). The score is an internal heuristic summary, and the code’s own docstring says so; any published threshold should be calibrated against the four realised values above.

S1.1.4 Contextual severity adjustment

Between a validator’s raw verdict and the routing decision sits

`ContextualSeverityAdjuster`, which can only ever *downgrade* a severity, never raise it.

Three rules are implemented (`guardian_core.py`), each with the source it is drawn from:

1. **Central-limit downgrade** — for `t_test`, `anova` and `pearson`, if the smallest group has $n \geq 30$, a critical normality violation becomes warning and a warning becomes minor (Lumley et al., 2002). This fires in S1.3.2.
2. **Balanced-design downgrade** — for `anova`, if $\max/\min$ group size ≤ 1.5 , a critical variance-homogeneity violation becomes warning (Box, 1954).
3. **Borderline- p downgrade** — for any violation carrying a p -value > 0.01 , a critical severity becomes warning. This fires in S1.3.1.

Every adjustment is written into the report’s `audit_trail` rather than applied silently. Executed on the S1.3.2 request, `context_adjustments_applied` is `True` and the normality entry carries `context_adjustment = "normality warning->minor (CLT, N>=30)"` alongside the validator’s methodological citation, so a reader of the report can see both that a downgrade happened and why.

S1.2 Programmatic access

Reproduced by the commands quoted below. All transcripts are verbatim output captured at the commit named in the provenance paragraph, against a locally running backend.

S1.2.1 Installation

The Python SDK and its `sfs` command-line entry point are published on PyPI:

```
$ python -m venv venv && ./venv/bin/pip install "stickforstats[cli]"
$ ./venv/bin/python -c "import stickforstats; print(stickforstats.__version__)"
0.4.0
$ ./venv/bin/sfs --version
sfs, version 0.4.0
```

The SDK is a thin REST client: it requires a reachable StickForStats backend (a local docker compose deployment, a manage.py runserver instance, or a hosted instance). The target is set by the `base_url` argument, the `STICKFORSTATS_BASE_URL` environment variable, or persistently with `sfs config`:

```
$ sfs config --base-url http://localhost:8000/api/v1
Configuration saved.
$ sfs config --show
StickForStats Configuration
```

Key	Value
base_url	http://localhost:8000/api/v1

The base URL must include the `/api/v1` prefix. Settings are written to `~/.stickforstats/config.json`.

S1.2.2 The Guardian cascade over the API

This is the routing mechanism of Fig. 2 exercised end to end. Two exponential samples with a $4\times$ variance ratio are generated under a stated seed, submitted as a t -test, and the returned `cascade_path` shows the assumption verdicts and the test the request was routed to:

```
import numpy as np
from stickforstats import StickForStats

SEED = 20260804
print("seed =", SEED, "| numpy", np.__version__)
rng = np.random.default_rng(SEED)
a = rng.exponential(scale=1.0, size=20).round(6).tolist() # skewed
b = rng.exponential(scale=2.0, size=20).round(6).tolist() # skewed + 4x
    variance

sfs = StickForStats(base_url="http://localhost:8000/api/v1")
```

```
r = sfs.post("autonomous/cascade/", json={"data": {"a": a, "b": b},
                                           "test": "t_test", "alpha": 0.05})

print("original_test :", r["original_test"])
print("final_test    :", r["final_test"])
print("n_cascades     :", r["n_cascades"])
for step in r["cascade_path"]:
    print(f"    {step['test']:<14} passed={step['passed']}")
    for v in step["violations"]:
        print(f"        violation: {v}")
    if step["alternatives"]:
        print(f"        alternatives: {step['alternatives']}")
```

Output:

```
seed = 20260804 | numpy 2.4.6
original_test : t_test
final_test    : mann_whitney
n_cascades    : 1
  t_test      passed=False
    violation: Normality assumption violated (p=0.0035)
    violation: Variance homogeneity violated (ratio=19.51, p=0.0009)
    violation: 12.5% of data are outliers
    alternatives: ['mann_whitney', 'permutation_test', 'bootstrap',
'welch_t_test']
  mann_whitney passed=True
```

Note that `autonomous/cascade/` returns the *routing decision* ("result": null); the selected test is then executed through its own endpoint.

S1.2.3 High-precision descriptive statistics

The backend computes descriptive statistics in extended precision. Applied to the control group of R's `PlantGrowth`:

```
from stickforstats import StickForStats
sfs = StickForStats(base_url="http://localhost:8000/api/v1")
d = sfs.stats.descriptive([4.17, 5.58, 5.18, 6.11, 4.50, 4.61, 5.17, 4.53, 5.33,
5.14])
print("sample_size :", d.sample_size)
print("precision    :", d.precision)
hp = d.high_precision_result
for k in ["mean", "median", "std", "variance"]:
    print(f"{k:<12}: {hp[k]}")
```

Output:

```
sample_size : 10
precision   : 50 decimal places
mean        : 5.032
median      : 5.155
std         : 0.5830913783924056234300667212541223916022435972595
variance    : 0.339995555555555555555555555555555555555555555555556
```

Independently verified with exact rational arithmetic (`fractions.Fraction`) plus `mpmath` at 60 decimal digits: the sample variance of these ten values is exactly **76499/225000 = 0.3399955555...**, and its square root to 50 digits is `0.58309137839240562343006672125412239160224359725949`. The returned `std` therefore agrees with the exact value to all digits shown, and the returned `variance` is the exact rational expansion. For comparison, R's double-precision `sd()` gives `0.58309137839240566` and `var()` gives `0.33999555555555561` — i.e. the extended-precision path is genuinely beyond double precision here, not decorated noise.

S1.2.4 A nonparametric test, cross-checked against R

```
from stickforstats import StickForStats
sfs = StickForStats(base_url="http://localhost:8000/api/v1")
# R's ToothGrowth, tooth length by supplement
OJ = [15.2, 21.5, 17.6, 9.7, 14.5, 10.0, 8.2, 9.4, 16.5, 9.7, 19.7, 23.3, 23.6,
      26.4, 20.0,
      25.2, 25.8, 21.2, 14.5, 27.3, 25.5, 26.4, 22.4, 24.5, 24.8, 30.9, 26.4,
      27.3, 29.4, 23.0]
VC = [4.2, 11.5, 7.3, 5.8, 6.4, 10.0, 11.2, 11.2, 5.2, 7.0, 16.5, 16.5, 15.2,
      17.3, 22.5,
      17.3, 13.6, 14.5, 18.8, 15.5, 23.6, 18.5, 33.9, 25.5, 26.4, 32.5, 26.7,
      21.5, 23.3, 29.5]
r = sfs.nonparametric.mann_whitney(group1=OJ, group2=VC)
hp = r.high_precision_result
print("test_name      :", hp["test_name"])
print("U statistic    :", hp["test_statistic"])
print("p_value        :", hp["p_value"])
print("sample_sizes   :", hp["sample_sizes"])
print("ties_present   :", hp["ties_present"])
```

Output:

```
test_name      : Mann-Whitney U Test
U statistic    : 575.5
p_value        : 0.0644906721338357
sample_sizes   : [30, 30]
ties_present   : True
```

R, independently: `wilcox.test(len ~ supp, data = ToothGrowth)` returns `W = 575.5`, `p-value = 0.06449` (with the same tie warning). Agreement to R's printed precision.

S1.2.5 Command-line manuscript checking

The CLI exposes the retrospective side of the platform. Given a short results paragraph:

```
$ cat example_results.txt
Results

Body mass was higher in the treated group than in controls,
t(28) = 2.10, p = 0.045. The omnibus comparison across the three
dose levels was significant, F(2, 27) = 4.85, p = 0.016. Expression
correlated with dose, r(48) = 0.35, p = 0.013. Genotype was
associated with response, chi2(1) = 6.10, p = 0.013.
```

`sfs verify` re-runs the reported statistics against any supplied raw data and reports a per-claim verdict. With no data attached, every claim resolves to `INSUFFICIENT_DATA`, which is the honest default:

```
$ sfs verify -f example_results.txt
Verification Summary
```

Metric	Value
Claims checked	3
Verifiability rate	0.0%
Coverage	100.0%
Citation-content conflicts	0
Run ID	(not persisted)

Verdict distribution

Verdict	Count
INSUFFICIENT_DATA	3

`sfs manuscript` runs the reporting-quality review (Statistical Quality Score, consistency recomputation, and JARS-style completeness findings). Selected fields from the returned JSON, verbatim:

```
$ sfs manuscript -f example_results.txt
{
  "word_count": 56,
  "sections_found": ["results"],
  "sqs_score": 20.0,
  "sqs_grade": "F",
  "consistency_rate": 1.0,
  "claims_found": 4,
  "claims_consistent": 3,
  "claims_inconsistent": 0,
  "decision_errors": 0,
  "gross_errors": 0,
  "overall_assessment": "major_issues",
  "findings": [
    {"severity": "moderate", "category": "reporting",
      "title": "Incomplete effect size reporting",
      "description": "Only 1 of 4 tests include effect sizes."},
    {"severity": "moderate", "category": "reporting",
      "title": "No confidence intervals reported"},
    {"severity": "moderate", "category": "methodology",
      "title": "Methods section not detected"},
    {"severity": "major", "category": "sqs",
      "title": "Poor Effect Sizes score (0%)"},
    {"severity": "major", "category": "sqs",
      "title": "Poor Assumption Transparency score (0%)"},
    {"severity": "major", "category": "sqs",
      "title": "Poor Sample and Power score (0%)"}
  ]
}
```

S1.2.6 Scope of the client at version 0.4.0

So that readers are not misled about what the published client currently covers: the transcripts above are the calls we executed successfully against the backend at this commit. Several other SDK/CLI surfaces in 0.4.0 send request bodies that the current backend serializers reject (`stats.ttest`, `stats.anova`, `stats.correlation`, `power.*`, `categorical.*`, `nonparametric.kruskal_wallis`, and the `sfs analyze` and `sfs profile` subcommands returned HTTP 400 or a client-side validation error in our runs). The full web application and the REST API itself are unaffected — the mismatch is in the thin client’s payload shapes — and realigning the SDK with the current serializers is tracked work. Readers wanting programmatic access to the endpoints not listed above should call the REST API directly, as in S1.2.2.

S1.3 Additional validation on standard R datasets

Regenerated by: `paper/replication/additional_file_1_evidence.py --only s1_3`

Three datasets from R’s `datasets` package were exported directly out of R 4.4.1 (so their provenance is R itself, not a hand-typed copy) and passed to the real `GuardianCore.check()`. SHA-256 of the exported CSVs:

Dataset	<i>n</i> rows	SHA-256 of exported CSV	bytes
mtcars	32	439ba335c3d28dd0c1871f75bdfb389d5a3b23cf703275566700140c9523ae5	1783
ToothGrowth	60	443c1ac5d1b7bec0cb5be80d6cd84abff169e4e6c2864f1e86bca7fc3b86749d	1054
PlantGrowth	30	22d383c8996e72b7db5eaa0d51c882a2cfdef1fb2aff999f80d81e3e5058c26b	518

No `observation_order` was declared for any of the three (all three are cross-sectional), so in every case the independence validator records `not_applicable` and refers the assumption to study design rather than returning a lag-1 verdict — the behaviour described in the Implementation section.

S1.3.1 `mtcars` — regression, `mpg ~ wt`

$n = 32$. `wt`: mean 3.217250, SD 0.978457. `mpg`: mean 20.090625, SD 6.026948. Reference fit (SciPy `linregress`): slope -5.3444715727 , intercept 37.2851261673 , $R^2 = 0.7528327937$, $p = 1.293959 \times 10^{-10}$.

`GuardianCore.check(data={"wt": ..., "mpg": ...}, test_type="regression")` returns:

Assumption	Result	Test performed	Statistic	<i>p</i>
normality	pass	Shapiro–Wilk	—	0.12288135948636324 (max over the two arrays)
independence	not applicable	referred to study design	—	—
homoscedasticity	pass	Breusch–Pagan	0.0404383955	0.8406258536969797
linearity	violation (warning)	Linearity check (residual analysis)	R^2 improvement 0.0662285644801448	0.048077016283035394 (Wald–Wolfowitz runs test)

- `confidence_score = 0.444`, `can_proceed = True`, `alternative_tests = []`.
- Violation message, verbatim: Linearity violated (R^2 improvement with polynomial: 0.066) (borderline: $p=0.0481$, close to threshold).
- The two-stage severity is visible here: the runs test detected a residual pattern ($p = 0.0481 < \alpha$), which raises a critical linearity violation, and the contextual severity adjuster then downgraded it to warning because $p > 0.01$. Independently, the R^2 improvement of 0.066 sits in the validator’s own 0.05–0.10 warning band. The substantive reading is that `mpg ~ wt` is mildly curved — which is a known feature of this dataset — and Guardian surfaces it rather than letting the linear fit pass silently.

S1.3.2 ToothGrowth — two-sample *t*-test, `len ~ supp`

OJ: $n = 30$, mean 20.663333, SD 6.605561. VC: $n = 30$, mean 16.963333, SD 8.266029. Reference test (SciPy `ttest_ind`): $t = 1.9152682687$, $p = 0.0603933712$.

`GuardianCore.check(data={"OJ": ..., "VC": ...}, test_type="t_test")` returns:

Assumption	Result	Test performed	Statistic	<i>p</i>
normality	violation (minor)	Shapiro–Wilk	$W = 0.9178433795430943$	0.023587729215913122
variance_homogeneity	pass	Levene’s test (median-centred)	—	0.2751764616144067
independence	not applicable	referred to study design	—	—
outliers	pass	IQR + z-score	—	—

- `confidence_score = 0.722`, `can_proceed = True`, `alternative_tests = ['mann_whitney', 'permutation_test', 'bootstrap']`.
- Violation message, verbatim: Normality assumption violated ($p=0.0236$) (downgraded: $N=30$ per group provides CLT robustness).
- The OJ group is the violating one (R: Shapiro $p = 0.0236$ for OJ, 0.4284 for VC). The raw severity was warning ($p > \alpha/10$); the contextual adjuster downgraded it to minor because $n = 30$ per group invokes central-limit robustness (Lumley et al., 2002). Because no violation is critical, `can_proceed` remains True and the parametric *t*-test runs, with the nonparametric alternatives offered rather than

imposed. Cross-checking Levene: `R car::leveneTest(center = median)` gives $F(1, 58) = 1.2136$, $p = 0.2752$.

- The same request submitted over the REST API through the published SDK (`autonomous/cascade/`, S1.2) returns the identical verdict: confidence 0.722, `can_proceed` True, one minor normality violation, `n_cascades` = 0 — i.e. the HTTP path and the direct call agree.

S1.3.3 PlantGrowth — one-way ANOVA, `weight ~ group`

`ctrl`: $n = 10$, mean 5.032000, SD 0.583091. `trt1`: $n = 10$, mean 4.661000, SD 0.793676. `trt2`: $n = 10$, mean 5.526000, SD 0.442573. Reference test (SciPy `f_oneway`): $F = 4.8460878624$, $p = 0.0159099583$.

`GuardianCore.check(data={"ctrl": ..., "trt1": ..., "trt2": ...}, test_type="anova")`
returns:

Assumption	Result	Test performed	p
normality	pass	Shapiro–Wilk	0.7474734451902609 (max over the three groups)
variance_homogeneity	pass	Levene’s test (median-centred)	0.34122662412547355
independence	not applicable	referred to study design	—
outliers	violation (critical)	Outlier detection (IQR + z-score)	—

- `confidence_score` = **0.167**, `can_proceed` = **False**, `alternative_tests` = [].
- Violation message, verbatim: 20.0% of data are outliers.
- Reproducing the detector’s arithmetic independently: only the `trt1` group has fenced values — 5.87 and 6.03 lie above its upper Tukey fence of 5.8637 ($Q1 = 4.2075$, $Q3 = 4.8700$) — giving $2/10 = 20.0\%$ for that group and 0% for `ctrl` and `trt2`. The reported figure is the mean percentage **across the groups that contain at least one outlier**, so 20.0% here describes one group, not the dataset: 2 of 30 observations, 6.67%, are fenced. Because the reported percentage exceeds the detector’s 10% critical threshold, the violation is `critical` and `can_proceed` becomes `False`.
- This case also exposes a real gap, reported here rather than smoothed over: for an ANOVA whose only critical violation is outliers, `alternative_tests` comes back **empty**, so the request is blocked without an alternative being offered. Guardian’s normality- and variance-driven ANOVA routes (Kruskal–Wallis, Welch’s ANOVA) do not cover the outlier-driven case. Cross-checks of the two passing validators against R: Shapiro $p = 0.7475$ (`ctrl`), 0.4519 (`trt1`), 0.5643 (`trt2`) — the reported value is the maximum, 0.7475; `car::leveneTest(center = median)` gives $F(2, 27) = 1.1192$, $p = 0.3412$.

S1.3.4 Summary

Dataset	Test	Confidence	Violations (severity)	Proceeds?
mtcars	regression <code>mpg ~ wt</code>	0.444	linearity (warning)	yes
ToothGrowth	two-sample <i>t</i> -test	0.722	normality (minor, CLT-downgraded)	yes
PlantGrowth	one-way ANOVA	0.167	outliers (critical)	no

Guardian raised a substantive assumption issue in all three of these textbook datasets while blocking only the one with a critical violation.

Note on `paper/replication/additional_real_data_analysis.py`. That script, which the manuscript previously cited for this section, does **not** call `GuardianCore`: it re-implements a few checks with bare SciPy on hand-typed copies of the three datasets, and it prints a summary asserting “all assumptions PASS” for all three — which, as the real Guardian output above shows, is not what Guardian returns. All numbers in S1.3 come from `paper/replication/additional_file_1_evidence.py`, which calls the real validator.

S1.4 Guardian test-suite coverage

Regenerated by: `paper/replication/additional_file_1_evidence.py --only s1_4`

Counts below are the test **runners**’ own totals (`manage.py test <module>` for the backend, `react-scripts test --testPathPattern=...` for the frontend), not `grep` counts of `def test_ / it()`, which are higher because not every defined test is collected.

Suite	File / module	Tests	Runner verdict
Guardian integration (backend)	backend/core/guardian/tests/test_guardian_integration.py	22	OK
Guardian middleware (backend)	backend/core/guardian/tests/test_guardian_middleware.py	16	OK
Guardian validator unit tests (backend)	backend/tests/test_guardian_validators.py	56	OK
Guardian math-correctness (backend)	backend/core/tests/test_guardian_math_fixes.py	12	OK
Guardian fail-loudly regression tests (backend)	backend/tests/test_guardian_fails_loudly.py	35	OK
Shape-similarity regression tests (backend)	backend/tests/test_guardian_similar_shapes.py	19	OK
Backend Guardian subtotal		160	
Guardian React components (frontend)	frontend/src/components/Guardian/__tests__/GuardianComponents.test.jsx	25	25 passed
useGuardianReport hook (frontend)	frontend/src/hooks/__tests__/useGuardianReport.test.js	30	30 passed
Frontend Guardian subtotal		55	

Whole-suite totals executed at the same commit: **backend 1,427 tests, OK** and **frontend 1,031 tests across 58 suites, all passing** — 2,458 tests in total.

This breakdown is not a complete accounting of the test suite. The eight suites above are the Guardian-specific ones; they are **160 of 1,427 backend tests (11.21%)** and **55 of 1,031 frontend tests (5.33%)**. The remaining ~90% of the backend and ~95% of the frontend suite cover the other platform modules (statistical endpoints, manuscript verification, genomics, power analysis, authentication, the React application) and are not enumerated here.

Reproducibility caveat, stated because it is the kind of discrepancy that otherwise looks like an error: the backend total is **1,427 on a clean checkout of the commit named above**. Run against a working tree that also contains uncommitted test files, the runner reports a larger number. The evidence script prints `git status --porcelain -- backend` immediately before the totals so this is always visible.

S1.5 Performance benchmarks

Regenerated by: `paper/replication/benchmark_api.py`

The manuscript's previous figure (a median Guardian overhead of 0.2 ms, with all latencies below 10 ms at the 99th percentile) was an artefact of the benchmark reusing one payload, so every measured request was served from the API's response cache. `benchmark_api.py` now draws a fresh random data set per request, interleaves the conditions on identical data in a seeded random order, treats any non-200 as a hard failure, and aborts if any measured response reports a cache hit. Across the run below, cache hits were **0 of 1700 measured requests**.

Endpoint	Condition	n	median (ms)	p95	p99	paired Δ vs standard (ms)	95% CI
t-test (independent)	standard	100	25.442	27.173	30.78	—	—
t-test (independent)	guardian	100	27.243	29.306	30.713	1.75	[1.588, 1.944]
t-test (independent)	validate	100	133.46	138.966	151.892	108.015	[107.522, 108.258]
t-test (independent)	both	100	135.1	144.451	166.686	109.552	[108.668, 109.99]
ANOVA (one-way)	standard	100	23.128	26.275	28.651	—	—
ANOVA (one-way)	guardian	100	25.494	28.019	30.433	2.407	[2.219, 2.567]
ANOVA (one-way)	validate	100	23.059	26.231	31.134	-0.044	[-0.276, 0.116]
ANOVA (one-way)	both	100	25.498	28.133	29.993	2.361	[2.242, 2.56]
Pearson correlation	standard	100	2.169	2.659	2.806	—	—
Pearson correlation	guardian	100	3.157	3.637	3.804	0.973	[0.941, 1.034]
Pearson correlation	validate	100	2.178	2.651	2.789	0.005	[-0.042, 0.047]
Pearson correlation	both	100	3.14	3.466	3.589	0.98	[0.931, 1.007]
Multiple linear regression	standard	100	120.479	131.945	140.974	—	—
Multiple linear regression	guardian	100	119.949	130.601	136.485	0.205	[-0.681, 0.623]
Multiple linear regression	validate	100	119.941	128.989	141.24	-0.176	[-0.778, 0.697]
Multiple linear regression	both	100	119.49	128.922	142.422	-0.535	[-1.285, -0.123]
Guardian pre-flight (/api/guardian/check/)	preflight	100	356.235	457.378	479.387	—	—

Read the paired deltas, not the absolute medians: absolute latency on a single-process development server drifts by roughly 10% between runs and with position in the run, whereas the condition contrasts are measured on identical data within the same iteration.

Three things this table says that the previous claim did not. The **in-endpoint assumption check is cheap** — 1.0 to 2.4 ms. The **standalone Guardian pre-flight call is not**: a median of about 356 ms, which a user clicking through the interface will feel, and which is the clearest performance target the platform currently has. And the **cross-language result validation is the dominant cost when requested** (+108 ms on the t-test) because it shells out to R, so it should stay opt-in.

Environment: `manage.py runserver 8000` (single process, `--noreload`),
`DJANGO_DEBUG=True`, Django 4.2.30, LocMemCache (Redis unavailable), macOS
26.5.2 arm64

References cited in this Additional file

Box GEP. Some theorems on quadratic forms applied in the study of analysis of variance problems. *Ann Math Stat.* 1954;25(2):290–302.

D’Agostino RB, Stephens MA, editors. *Goodness-of-fit techniques*. New York: Marcel Dekker; 1986.

Lumley T, Diehr P, Emerson S, Chen L. The importance of the normality assumption in large public health data sets. *Annu Rev Public Health.* 2002;23:151–69.

R Core Team. *R: a language and environment for statistical computing*. Version 4.4.1. Vienna: R Foundation for Statistical Computing; 2024. The `mtcars`, `ToothGrowth` and `PlantGrowth` datasets are distributed in the base `datasets` package.